

Phase 5: Priority Queues and Full C Implementation — Full Structured Beginner Notes

1. Current Progress

You have already completed the first four phases of the heap curriculum.

What you learned in Phase 1

You learned the basic foundation of heaps:

- A heap is a special binary tree.
- A heap must be a complete binary tree.
- A heap is usually stored in an array.
- In this curriculum, index `0` is ignored.
- The root starts at index `1`.
- For a node at index `i`:
- Left child = `2 * i`
- Right child = `2 * i + 1`
- Parent = `i / 2`
- A max heap keeps the largest value at the root.
- A min heap keeps the smallest value at the root.
- A heap is not fully sorted.
- A heap only guarantees correct parent-child order.

What you learned in Phase 2

You learned insertion.

The main idea of insertion is:

Insert at the next free position first, then shift up.

Insertion moves:

Bottom → Top
Leaf → Root

During insertion, we compare the inserted value with its parent.

For a max heap:

If child > parent, the child moves upward.

Insertion takes:

$O(\log N)$

because the value moves only through the height of the heap.

What you learned in Phase 3

You learned deletion and heap sort.

The main idea of deletion is:

Delete the root, move the last element to the root, then shift down.

Deletion moves:

Top → Bottom
Root → Leaf

During deletion, we compare the moved value with its children.

For a max heap:

Choose the larger child during shift-down.

You also learned that heap sort is based on repeated deletion.

What you learned in Phase 4

You learned heapify.

Heapify is a faster way to build a heap from an unsorted array.

Repeated insertion takes:

$O(N \log N)$

Heapify takes:

$O(N)$

Heapify works by:

Start from $n / 2$.
Move backward to index 1.
Apply `shiftDown` at each internal node.

Important memory rule:

Insertion moves upward.
Deletion moves downward.
Heapify also moves downward.

Now Phase 5 connects all heap operations into a useful real-world data structure:

Priority Queue

2. What This Phase Covers

In this phase, you will learn:

1. What a priority queue is.
2. How a priority queue is different from a normal queue.
3. Why heaps are used to implement priority queues.
4. What a max-priority queue is.
5. What a min-priority queue is.
6. The main operations of a priority queue.
7. How insertion works in a priority queue.
8. How `peekMax()` works.
9. How `extractMax()` works.
10. The difference between `peekMax()` and `extractMax()`.
11. Full C implementation of a max-priority queue.
12. Dry run of priority queue insertion.
13. Dry run of `extractMax()`.
14. Time complexity of priority queue operations.
15. How to convert the code into a min-priority queue.
16. Common beginner mistakes.
17. Important rules to memorize.
18. Practice questions and answers.

3. Big Idea of Phase 5

A heap is useful because the most important value is always at the root.

For a max heap:

The largest value is always at A[1].

For a min heap:

The smallest value is always at A[1].

A priority queue is a data structure where the most important item comes out first.

So heaps and priority queues fit together naturally.

Simple meaning:

Heap = keeps most important value at the root.
Priority queue = removes most important value first.

That is why heaps are commonly used to implement priority queues.

4. What Is a Normal Queue?

A normal queue works like a line of people.

The first person who enters the line is the first person who leaves.

This rule is called:

FIFO

FIFO means:

First In, First Out

Example:

```
Queue: Ali, Sara, Omar
```

Ali came first, so Ali leaves first.

Then Sara leaves.

Then Omar leaves.

So the order is:

```
Ali → Sara → Omar
```

A normal queue cares about arrival order.

5. What Is a Priority Queue?

A priority queue does not always remove the first item that arrived.

Instead, it removes the item with the highest priority.

Simple meaning:

```
Priority queue = remove by importance, not by arrival order.
```

Example:

```
Patients in a hospital:  
Patient A: mild cold  
Patient B: broken arm  
Patient C: heart attack
```

Even if Patient A arrived first, Patient C should be treated first.

Why?

Because Patient C has the highest priority.

So the priority queue removes Patient C first.

6. Normal Queue vs Priority Queue

Feature	Normal Queue	Priority Queue
Removal rule	First item comes out first	Highest-priority item comes out first
Main idea	Arrival order	Importance order
Example	People waiting in a line	Hospital emergency system
Common rule	FIFO	Priority-based

Simple memory:

Normal queue asks: Who came first?
Priority queue asks: Who is most important?

7. Why Do We Use a Heap for a Priority Queue?

A priority queue needs quick access to the most important item.

A heap gives quick access to the most important item because the root stores it.

In a max heap:

$A[1] = \text{largest value}$

If a larger number means higher priority, then the highest priority is always at $A[1]$.

In a min heap:

$A[1] = \text{smallest value}$

If a smaller number means higher priority, then the highest priority is always at $A[1]$.

So a heap helps us avoid scanning the whole array to find the most important item.

Without a heap, we might need:

$O(N)$

time to find the highest-priority item.

With a heap, we can look at the root in:

$O(1)$

8. Max-Priority Queue

A max-priority queue uses a max heap.

In a max-priority queue:

Higher number = higher priority

Example:

Priority values: 20, 15, 40, 10, 30

The highest priority is:

40

So 40 should come out first.

A max heap makes this easy because the largest value is always at the root.

Example max heap:

Index:	1	2	3	4	5
Value:	40	30	20	10	15

Here:

$A[1] = 40$

So the highest-priority value is immediately available.

9. Min-Priority Queue

A min-priority queue uses a min heap.

In a min-priority queue:

Smaller number = higher priority

Example:

Priority values: 20, 15, 40, 10, 30

If smaller value means more urgent, then the highest priority is:

10

So 10 should come out first.

A min heap makes this easy because the smallest value is always at the root.

Example min heap:

Index: 1 2 3 4 5
Value: 10 15 40 20 30

Here:

A[1] = 10

So the highest-priority value is immediately available.

10. Max-Priority Queue vs Min-Priority Queue

Feature	Max-Priority Queue	Min-Priority Queue
Heap type	Max heap	Min heap
Root contains	Largest value	Smallest value
Higher priority means	Bigger value	Smaller value

Feature	Max-Priority Queue	Min-Priority Queue
Example first removed	40 from [20, 15, 40]	15 from [20, 15, 40]
Used when	Bigger number is more important	Smaller number is more urgent

Simple memory:

Max-priority queue: biggest comes out first.
Min-priority queue: smallest comes out first.

11. Main Operations of a Priority Queue

A priority queue usually supports these operations:

Operation	Meaning	Heap action
<code>insert(value)</code>	Add a new priority value	Insert at end, then shift up
<code>peekMax()</code>	See the highest priority without removing it	Return <code>A[1]</code>
<code>extractMax()</code>	Remove and return the highest priority	Delete root, then shift down
<code>isEmpty()</code>	Check whether the queue has no elements	Check size
<code>isFull()</code>	Check whether the array is full	Check capacity
<code>display()</code>	Print the heap array	Visit each active element

The two most important operations are:

`insert`
`extractMax`

Why?

Because a priority queue mainly needs to add items and remove the highest-priority item.

12. Beginner Analogy

Think of a priority queue as a competition podium.

In a max heap:

The strongest player must stay at the top.

When a new player enters:

The player starts at the bottom.
If the player is stronger than their parent, they climb upward.

When the top player leaves:

The last player moves to the top.
Then the player falls downward until the order is correct again.

This is exactly how a heap-based priority queue works.

13. Insert in a Priority Queue

Insertion in a priority queue uses the same logic as heap insertion from Phase 2.

The new value is first placed at the next free position.

Then it may move upward.

For a max-priority queue:

If the new value is bigger than its parent, it moves upward.

Insertion steps

1. Increase the heap size.
2. Put the new value at the next free position.
3. Compare it with its parent.
4. If it is bigger than the parent, move the parent down.
5. Move upward to the parent index.
6. Repeat until the value reaches the correct position.

Simple memory:

Priority queue insertion = heap insertion = shift up.

14. Insert Example: Insert 50

Start with this max heap:

Index:	1	2	3	4
Value:	40	20	30	10

Now insert:

50

First, put 50 at the next free position.

The next free position is index 5:

Index:	1	2	3	4	5
Value:	40	20	30	10	50

Now compare 50 with its parent.

Parent index:

$5 / 2 = 2$

Parent value:

$A[2] = 20$

Compare:

$50 > 20$

So 50 must move upward.

Now 50 reaches index 2.

Compare with its new parent.

Parent index:

$$2 / 2 = 1$$

Parent value:

`A[1] = 40`

Compare:

`50 > 40`

So `50` moves upward again.

Final heap:

Index:	1	2	3	4	5
Value:	50	40	30	10	20

Now the highest-priority value is at the root.

15. `peekMax()` Operation

`peekMax()` means:

Look at the highest-priority value without removing it.

For a max heap, the highest-priority value is always at:

`A[1]`

So `peekMax()` simply returns:

```
return A[1];
```

Example:

Heap: 40, 20, 30, 10

`peekMax()` returns:

40

After `peekMax()`, the heap is still:

40, 20, 30, 10

Nothing is removed.

Nothing is shifted.

The heap does not change.

16. `extractMax()` Operation

`extractMax()` means:

Remove and return the highest-priority value.

In a max heap, the highest-priority value is at:

A[1]

So `extractMax()` removes the root.

This uses the same idea as heap deletion from Phase 3.

`extractMax()` steps

1. Check whether the priority queue is empty.
2. Save the root value.
3. Move the last element to the root.
4. Reduce the heap size.
5. Shift down from the root.
6. Return the saved root value.

Simple memory:

```
extractMax = delete root + shift down
```

17. Extract Max Example

Start with this heap:

Index:	1	2	3	4	5
Value:	50	40	30	10	20

The highest-priority value is:

50

Step 1: Save the root

```
maxValue = 50
```

Step 2: Move the last value to the root

The last value is:

20

Move 20 to index 1:

Index:	1	2	3	4
Value:	20	40	30	10

Now the shape is correct, but the order may be wrong.

Step 3: Shift down

Children of 20 are:

40 and 30

Choose the larger child:

40

Compare:

40 > 20

So swap 20 and 40.

Final heap:

```
Index: 1  2  3  4
Value: 40 20 30 10
```

Now the heap is valid again.

So `extractMax()` returns:

50

The new highest-priority value is:

40

18. `peekMax()` vs `extractMax()`

Operation	Removes value?	Changes heap?	Returns
<code>peekMax()</code>	No	No	Root value
<code>extractMax()</code>	Yes	Yes	Root value

Simple memory:

```
peek = look only
extract = remove and return
```

Example:

Heap:

40, 30, 20, 10

peekMax() returns:

40

Heap after peekMax():

40, 30, 20, 10

extractMax() returns:

40

Heap after extractMax():

30, 10, 20

19. Full C Implementation: Max Priority Queue

```
#include <stdio.h>

#define MAX 100

typedef struct {
    int A[MAX];
    int size;
} PriorityQueue;

void init(PriorityQueue *pq) {
    pq->size = 0;
    pq->A[0] = 0; // index 0 is ignored
}

int isEmpty(PriorityQueue *pq) {
    return pq->size == 0;
}
```



```

int isFull(PriorityQueue *pq) {
    return pq->size == MAX - 1;
}

void shiftUp(PriorityQueue *pq, int i) {
    int temp = pq->A[i];

    while (i > 1 && temp > pq->A[i / 2]) {
        pq->A[i] = pq->A[i / 2];
        i = i / 2;
    }

    pq->A[i] = temp;
}

void insert(PriorityQueue *pq, int value) {
    if (isFull(pq)) {
        printf("Priority queue is full.\n");
        return;
    }

    pq->size++;
    pq->A[pq->size] = value;

    shiftUp(pq, pq->size);
}

void shiftDown(PriorityQueue *pq, int i) {
    int j = 2 * i;

    while (j <= pq->size) {
        if (j < pq->size && pq->A[j + 1] > pq->A[j]) {
            j = j + 1;
        }

        if (pq->A[i] < pq->A[j]) {
            int temp = pq->A[i];
            pq->A[i] = pq->A[j];
            pq->A[j] = temp;

            i = j;
            j = 2 * i;
        } else {
            break;
        }
    }
}

```

```

int peekMax(PriorityQueue *pq) {
    if (isEmpty(pq)) {
        printf("Priority queue is empty.\n");
        return -1;
    }

    return pq->A[1];
}

int extractMax(PriorityQueue *pq) {
    if (isEmpty(pq)) {
        printf("Priority queue is empty.\n");
        return -1;
    }

    int maxValue = pq->A[1];

    pq->A[1] = pq->A[pq->size];
    pq->size--;

    shiftDown(pq, 1);

    return maxValue;
}

void display(PriorityQueue *pq) {
    if (isEmpty(pq)) {
        printf("Priority queue is empty.\n");
        return;
    }

    for (int i = 1; i <= pq->size; i++) {
        printf("%d ", pq->A[i]);
    }

    printf("\n");
}

int main() {
    PriorityQueue pq;
    init(&pq);

    insert(&pq, 20);
    insert(&pq, 15);
    insert(&pq, 40);
    insert(&pq, 10);
    insert(&pq, 30);

```

```

    printf("Priority queue as heap: ");
    display(&pq);

    printf("Highest priority: %d\n", peekMax(&pq));

    printf("Extracted: %d\n", extractMax(&pq));

    printf("After extractMax: ");
    display(&pq);

    return 0;
}

```

20. Explanation of the Full C Program

```
#define MAX 100
```

This sets the maximum size of the priority queue array.

```
#define MAX 100
```

Since index `0` is ignored, the usable positions are:

```
1 to 99
```

So the maximum number of stored values is `MAX - 1`.

PriorityQueue structure

```

typedef struct {
    int A[MAX];
    int size;
} PriorityQueue;

```

This structure stores two things:

Part	Meaning
<code>A[MAX]</code>	The heap array

Part	Meaning
<code>size</code>	Number of active elements

The heap values are stored from index `1` to index `size`.

`init()` function

```
void init(PriorityQueue *pq) {  
    pq->size = 0;  
    pq->A[0] = 0;  
}
```

This prepares the priority queue before using it.

It sets:

```
size = 0
```

That means the priority queue is empty.

It also sets:

```
A[0] = 0
```

But `A[0]` is ignored in this curriculum.

`isEmpty()` function

```
int isEmpty(PriorityQueue *pq) {  
    return pq->size == 0;  
}
```

This checks whether the priority queue has no elements.

If:

```
size == 0
```

then the priority queue is empty.

`isFull()` function

```
int isFull(PriorityQueue *pq) {  
    return pq->size == MAX - 1;  
}
```

This checks whether the priority queue is full.

Why `MAX - 1`?

Because index `0` is ignored.

So if `MAX = 100`, the usable indexes are:

1 to 99

21. Explanation of `shiftUp()`

```
void shiftUp(PriorityQueue *pq, int i) {  
    int temp = pq->A[i];  
  
    while (i > 1 && temp > pq->A[i / 2]) {  
        pq->A[i] = pq->A[i / 2];  
        i = i / 2;  
    }  
  
    pq->A[i] = temp;  
}
```

`shiftUp()` fixes the heap after insertion.

The new value starts at the bottom.

If it is bigger than its parent, it moves upward.

Important variables

Variable	Meaning
<code>pq</code>	The priority queue
<code>i</code>	Current index of the inserted value
<code>temp</code>	Stores the inserted value safely
<code>pq->A[i / 2]</code>	Parent value

Why use `temp`?

We use `temp` to safely hold the inserted value.

Simple meaning:

`temp` is like holding the new value in your hand while moving parents down.

If we do not use `temp`, the inserted value may be overwritten.

Loop condition

```
while (i > 1 && temp > pq->A[i / 2])
```

This means the loop continues while:

1. We are not at the root.
2. The inserted value is bigger than its parent.

If both are true, the inserted value should move upward.

22. Explanation of `insert()`

```
void insert(PriorityQueue *pq, int value) {
    if (isFull(pq)) {
        printf("Priority queue is full.\n");
        return;
    }

    pq->size++;
    pq->A[pq->size] = value;
```

```
    shiftUp(pq, pq->size);  
}
```

This function adds a new value to the priority queue.

Step 1: Check if full

```
if (isFull(pq))
```

If the priority queue is full, insertion is not possible.

Step 2: Increase size

```
pq->size++;
```

This creates the next free position.

Step 3: Place value at the end

```
pq->A[pq->size] = value;
```

The new value is inserted at the next free position.

This preserves the complete binary tree shape.

Step 4: Shift up

```
shiftUp(pq, pq->size);
```

This restores the max heap order.

23. Explanation of `shiftDown()`

```
void shiftDown(PriorityQueue *pq, int i) {  
    int j = 2 * i;  
  
    while (j <= pq->size) {  
        if (j < pq->size && pq->A[j + 1] > pq->A[j]) {  
            j = j + 1;  
        }  
    }  
}
```

```

        if (pq->A[i] < pq->A[j]) {
            int temp = pq->A[i];
            pq->A[i] = pq->A[j];
            pq->A[j] = temp;

            i = j;
            j = 2 * i;
        } else {
            break;
        }
    }
}

```

`shiftDown()` fixes the heap after removing the root.

After `extractMax()`, the last element is moved to the root.

That value may be too small, so it must move downward.

Important variables

Variable	Meaning
<code>i</code>	Current parent index
<code>j</code>	Child index
<code>pq->A[i]</code>	Parent value
<code>pq->A[j]</code>	Larger child value

Start with left child

```
int j = 2 * i;
```

Because:

```
Left child = 2 * i
```

Continue while child exists

```
while (j <= pq->size)
```


This means the loop continues while the current node has at least a left child.

Choose the larger child

```
if (j < pq->size && pq->A[j + 1] > pq->A[j]) {  
    j = j + 1;  
}
```

This checks if the right child exists and is bigger than the left child.

If yes, `j` moves to the right child.

After this step:

`j` points to the larger child.

Swap if parent is smaller

```
if (pq->A[i] < pq->A[j])
```

If the parent is smaller than the larger child, the max heap rule is broken.

So we swap them.

Then we continue shifting down.

24. Explanation of `peekMax()`

```
int peekMax(PriorityQueue *pq) {  
    if (isEmpty(pq)) {  
        printf("Priority queue is empty.\n");  
        return -1;  
    }  
  
    return pq->A[1];  
}
```

This function returns the highest-priority value without removing it.

If the queue is empty, it prints a message and returns `-1`.

If the queue is not empty, it returns:

```
pq->A[1]
```

Because the root contains the maximum value.

25. Explanation of `extractMax()`

```
int extractMax(PriorityQueue *pq) {
    if (isEmpty(pq)) {
        printf("Priority queue is empty.\n");
        return -1;
    }

    int maxValue = pq->A[1];

    pq->A[1] = pq->A[pq->size];
    pq->size--;

    shiftDown(pq, 1);

    return maxValue;
}
```

This function removes and returns the highest-priority value.

Step 1: Check if empty

```
if (isEmpty(pq))
```

If the priority queue is empty, we cannot remove anything.

Step 2: Save the root

```
int maxValue = pq->A[1];
```

The root is the maximum value, so we save it.

Step 3: Move last element to root

```
pq->A[1] = pq->A[pq->size];
```

This fills the root position.

Step 4: Reduce size

```
pq->size--;
```

The heap now has one fewer active element.

Step 5: Shift down

```
shiftDown(pq, 1);
```

This restores the max heap rule.

Step 6: Return removed value

```
return maxValue;
```

The function returns the extracted maximum.

26. Explanation of `display()`

```
void display(PriorityQueue *pq) {
    if (isEmpty(pq)) {
        printf("Priority queue is empty.\n");
        return;
    }

    for (int i = 1; i <= pq->size; i++) {
        printf("%d ", pq->A[i]);
    }

    printf("\n");
}
```

This function prints all active heap values.

It starts at index `1` because index `0` is ignored.

It stops at `pq->size` because values after `size` are not part of the active heap.

Important:

The printed heap array is not necessarily sorted.

It only follows the heap rule.

27. Dry Run of the Program

The program inserts:

20, 15, 40, 10, 30

We will trace each insertion.

Insert 20

The queue is empty.

Insert `20` at index `1`.

Heap:

[20]

No shift-up is needed because `20` is already the root.

Insert 15

Insert `15` at index `2`.

Heap before checking:

[20, 15]

Parent of index 2:

$$2 / 2 = 1$$

Parent value:

20

Compare:

15 > 20? No

So 15 stays where it is.

Heap:

[20, 15]

Insert 40

Insert 40 at index 3.

Heap before checking:

[20, 15, 40]

Parent of index 3:

$$3 / 2 = 1$$

Parent value:

20

Compare:

40 > 20

So 40 moves upward.

Heap becomes:

[40, 15, 20]

Now 40 is the root.

Insert 10

Insert 10 at index 4.

Heap before checking:

[40, 15, 20, 10]

Parent of index 4:

$4 / 2 = 2$

Parent value:

15

Compare:

10 > 15? No

So 10 stays where it is.

Heap:

[40, 15, 20, 10]

Insert 30

Insert 30 at index 5.

Heap before checking:

[40, 15, 20, 10, 30]

Parent of index 5:

$5 / 2 = 2$

Parent value:

15

Compare:

$30 > 15$

So 30 moves upward.

Heap becomes:

[40, 30, 20, 10, 15]

Now the highest-priority value is:

40

28. Dry Run of peekMax()

Current heap:

[40, 30, 20, 10, 15]

peekMax() returns:

```
A[1] = 40
```

The heap does not change.

After `peekMax()`:

```
[40, 30, 20, 10, 15]
```

29. Dry Run of `extractMax()`

Current heap:

```
[40, 30, 20, 10, 15]
```

Step 1: Save root

```
maxValue = 40
```

Step 2: Move last element to root

Last element:

```
15
```

Move `15` to the root:

```
[15, 30, 20, 10]
```

Step 3: Shift down

Children of `15`:

```
30 and 20
```

Larger child:

30

Compare:

30 > 15

Swap:

[30, 15, 20, 10]

Now 15 is at index 2.

Its child is:

10

Compare:

10 > 15? No

So shifting stops.

Final heap:

[30, 15, 20, 10]

extractMax() returns:

40

30. Expected Program Output

The exact output may look like this:

```
Priority queue as heap: 40 30 20 10 15
Highest priority: 40
```

Extracted: 40
After extractMax: 30 15 20 10

Important:

40 30 20 10 15

is a heap, not a fully sorted array.

It is valid because parent-child relationships are correct.

31. Complexity of Priority Queue Operations

Operation	Time Complexity	Reason
<code>isEmpty()</code>	$O(1)$	Only checks <code>size</code>
<code>isFull()</code>	$O(1)$	Only checks <code>size</code>
<code>peekMax()</code>	$O(1)$	Maximum is directly at <code>A[1]</code>
<code>insert()</code>	$O(\log N)$	New value may shift up through heap height
<code>extractMax()</code>	$O(\log N)$	Root replacement may shift down through heap height
<code>display()</code>	$O(N)$	Prints every active element

Important idea:

Priority queue is fast because the highest-priority value is always at the root.

32. Space Complexity

The priority queue uses an array.

If the capacity is `MAX`, the array uses:

$O(\text{MAX})$

If we describe it based on the number of elements, we usually say:

$O(N)$

The operations themselves use only a few extra variables.

So extra working space for operations is:

$O(1)$

33. Why `peekMax()` Is $O(1)$

`peekMax()` is very fast because it does not search.

It directly returns the root:

`A[1]`

So no matter how many elements are in the heap, the operation takes constant time.

Example:

`N = 10` $\rightarrow$ `A[1]` gives max
`N = 1000` $\rightarrow$ `A[1]` gives max
`N = 100000` $\rightarrow$ `A[1]` gives max

The root is always available immediately.

So:

`peekMax =  $O(1)$`

34. Why `insert()` Is $O(\log N)$

Insertion places the new value at the next free position.

Then it may shift upward.

The value moves along one path:

Leaf → Parent → Parent → Root

The length of this path is the height of the heap.

A heap is a complete binary tree, so its height is:

$O(\log N)$

Therefore:

insert = $O(\log N)$

35. Why `extractMax()` Is $O(\log N)$

`extractMax()` removes the root.

Then it moves the last element to the root.

That value may shift downward.

The value moves along one path:

Root → Child → Child → Leaf

The length of this path is the height of the heap.

Heap height is:

$O(\log N)$

Therefore:

extractMax = $O(\log N)$

36. How to Convert to a Min-Priority Queue

The current implementation is a max-priority queue.

That means:

Bigger value = higher priority

To create a min-priority queue:

Smaller value = higher priority

So we use a min heap instead of a max heap.

Change 1: `shiftUp()` comparison

For a max heap, we use:

```
temp > pq->A[i / 2]
```

For a min heap, use:

```
temp < pq->A[i / 2]
```

This means smaller values move upward.

Change 2: `shiftDown()` child choice

For a max heap:

Choose the larger child.

For a min heap:

Choose the smaller child.

Change 3: function names

For a min-priority queue, names usually become:

```
peekMin()  
extractMin()
```

instead of:

```
peekMax()  
extractMax()
```

37. Max Heap vs Min Heap Priority Queue Logic

Operation idea	Max-priority queue	Min-priority queue
Root contains	Maximum value	Minimum value
Insert movement	Bigger value moves up	Smaller value moves up
Shift-up condition	<code>temp > parent</code>	<code>temp < parent</code>
Shift-down child choice	Larger child	Smaller child
Remove operation	<code>extractMax()</code>	<code>extractMin()</code>

Simple memory:

```
Max heap pushes big values upward.  
Min heap pushes small values upward.
```

38. Important Variables in Phase 5

Variable	Meaning
<code>A[]</code>	Array storing the heap
<code>MAX</code>	Maximum array capacity
<code>size</code>	Number of active elements
<code>pq</code>	Pointer to the priority queue
<code>i</code>	Current index during shift-up or shift-down
<code>j</code>	Child index during shift-down

Variable	Meaning
<code>temp</code>	Temporary value used during movement or swap
<code>value</code>	New value being inserted
<code>maxValue</code>	Root value removed by <code>extractMax()</code>
<code>A[1]</code>	Root of the heap

39. Direction Comparison

Operation	Starts From	Moves Toward	Compares With
Insertion	New leaf	Root	Parent
Deletion / <code>extractMax()</code>	Root	Leaf	Children
Heapify	Internal node	Leaf	Children
<code>peekMax()</code>	Root	Nowhere	Nothing

Simple memory:

```
insert climbs up
extract falls down
peek only looks
```

40. Common Beginner Mistakes

Mistake 1: Thinking a priority queue is a normal queue

Wrong idea:

```
The first inserted value must be removed first.
```

Correct idea:

```
The highest-priority value is removed first.
```

A priority queue cares about importance, not arrival order.

Mistake 2: Searching manually for the highest priority

Wrong idea:

Scan the whole array to find the maximum.

Correct idea:

In a max heap, the maximum is always at $A[1]$.

That is why `peekMax()` is $O(1)$.

Mistake 3: Forgetting to shift up after insertion

Wrong idea:

Place the new value at the end and stop.

Problem:

The heap order may be broken.

Correct idea:

Insert at the end, then shift up.

Mistake 4: Forgetting to shift down after extraction

Wrong idea:

Move the last element to the root and stop.

Problem:

The root may be smaller than its children.

Correct idea:

Move the last element to the root, then shift down.

Mistake 5: Confusing `peekMax()` and `extractMax()`

Wrong idea:

`peekMax` removes the maximum.

Correct idea:

`peekMax` only looks.
`extractMax` removes.

Mistake 6: Expecting the priority queue array to be sorted

Wrong idea:

The heap array should be fully sorted.

Correct idea:

The heap array only follows parent-child order.

Example:

40, 30, 20, 10, 15

This is not fully sorted, but it is a valid max heap.

Mistake 7: Forgetting index 0 is ignored

This curriculum uses 1-based indexing.

So the root is:

A[1]

not:

A[0]

The formulas work because index 0 is ignored.

Mistake 8: Returning A[1] without checking empty queue

Wrong idea:

```
return pq->A[1];
```

without checking if the queue is empty.

Problem:

If the queue is empty, A[1] does not contain a valid priority value.

Correct idea:

```
if (isEmpty(pq)) {  
    printf("Priority queue is empty.\n");  
    return -1;  
}
```

Mistake 9: Choosing the wrong child during shift-down

For a max heap, choose the larger child.

Wrong idea:

Always choose the left child.

Correct idea:

Compare left and right child first.
Choose the larger child.

For a min heap, choose the smaller child.

41. Key Terms

Term	Simple Meaning
Priority queue	A structure that removes the most important item first
Priority	Importance level of an item
FIFO	First In, First Out
Max-priority queue	Priority queue where bigger value means higher priority
Min-priority queue	Priority queue where smaller value means higher priority
Max heap	Heap where parent is greater than or equal to children
Min heap	Heap where parent is less than or equal to children
<code>insert()</code>	Adds a new value
<code>peekMax()</code>	Returns maximum without removing it
<code>extractMax()</code>	Removes and returns maximum
<code>shiftUp()</code>	Moves inserted value upward
<code>shiftDown()</code>	Moves root replacement downward
<code>isEmpty()</code>	Checks whether queue has no elements
<code>isFull()</code>	Checks whether array capacity is full
<code>size</code>	Number of active heap elements
Root	The first active heap element at <code>A[1]</code>

42. Important Rules to Memorize

Rule 1

A priority queue removes by priority, not by arrival order.

Rule 2

A max-priority queue uses a max heap.

Rule 3

In a max-priority queue, the highest-priority value is at:

A[1]

Rule 4

`peekMax()` only looks at the root.

Rule 5

`extractMax()` removes the root.

Rule 6

Insertion uses shift-up.

Rule 7

Extraction uses shift-down.

Rule 8

In max heap shift-down, choose the larger child.

Rule 9

The priority queue array is not fully sorted.

Rule 10

The heap only guarantees parent-child order.

43. Simple Practice Questions

Use this max-priority queue:

Index:	1	2	3	4	5	6	7
Value:	50	30	40	10	20	35	25

Questions:

1. What value is at the root?
2. What value does `peekMax()` return?
3. Does `peekMax()` remove the value?
4. What value does `extractMax()` remove first?
5. After removing the root, which value moves to the root first?
6. During shift-down in a max heap, do we choose the larger child or smaller child?
7. What is the time complexity of `peekMax()`?
8. What is the time complexity of `insert()`?
9. What is the time complexity of `extractMax()`?
10. Is the heap array fully sorted?
11. What is the main difference between a normal queue and a priority queue?
12. How do we change max-priority queue insertion into min-priority queue insertion?

44. Practice Answers

1. What value is at the root?

The root is at index `1`.

`A[1] = 50`

Answer:

50

2. What value does `peekMax()` return?

`peekMax()` returns the root value.

`A[1] = 50`

Answer:

50

3. Does `peekMax()` remove the value?

No.

`peekMax()` only looks at the highest-priority value.

Answer:

No, it does not remove the value.

4. What value does `extractMax()` remove first?

`extractMax()` removes the root.

The root is:

50

Answer:

50

5. After removing the root, which value moves to the root first?

The last active element moves to the root.

The last active element is:

A[7] = 25

Answer:

25

6. During shift-down in a max heap, do we choose the larger child or smaller child?

In a max heap, the larger value should move upward.

Answer:

Choose the larger child.

7. What is the time complexity of `peekMax()`?

`peekMax()` directly returns `A[1]`.

Answer:

$O(1)$

8. What is the time complexity of `insert()`?

Insertion may shift up through heap height.

Heap height is $O(\log N)$.

Answer:

$O(\log N)$

9. What is the time complexity of `extractMax()`?

Extraction may shift down through heap height.

Heap height is $O(\log N)$.

Answer:

$O(\log N)$

10. Is the heap array fully sorted?

Array:

50, 30, 40, 10, 20, 35, 25

This is not fully sorted.

But it is a valid max heap because parent-child order is correct.

Answer:

No, it is not fully sorted.

11. What is the main difference between a normal queue and a priority queue?

A normal queue removes the first inserted item first.

A priority queue removes the highest-priority item first.

Answer:

Normal queue = arrival order.
Priority queue = importance order.

12. How do we change max-priority queue insertion into min-priority queue insertion?

For max heap insertion, bigger values move upward:

$\text{temp} > \text{parent}$

For min heap insertion, smaller values move upward:


```
temp < parent
```

Answer:

```
Change the shift-up comparison from > to <.
```

45. Final Phase 5 Summary

Phase 5 teaches how heaps become useful in real programs.

The main structure in this phase is:

```
Priority Queue
```

A priority queue removes the most important item first.

A heap is a good way to implement a priority queue because the most important item is always at the root.

For a max-priority queue:

```
Highest priority = largest value  
Root = maximum value
```

The main operations are:

```
insert(value)  
peekMax()  
extractMax()  
isEmpty()  
isFull()  
display()
```

`insert(value)`:

```
Add value at the next free position, then shift up.
```

`peekMax()`:

Return A[1] without removing it.

`extractMax()`:

Remove A[1], move last element to root, reduce size, then shift down.

Complexities:

<code>peekMax</code>	<code>= O(1)</code>
<code>insert</code>	<code>= O(log N)</code>
<code>extractMax</code>	<code>= O(log N)</code>
<code>isEmpty</code>	<code>= O(1)</code>
<code>isFull</code>	<code>= O(1)</code>
<code>display</code>	<code>= O(N)</code>

Most important beginner memory:

Priority queue uses heap rules to always keep the most important value at the root.

Another memory rule:

insert climbs up
extract falls down
peek only looks

You now understand how heaps can be used as a complete priority queue implementation.